

第四章 数据准备

预训练是研发大语言模型的第一个训练阶段，也是最为重要的一个阶段。有效的预训练能够为大语言模型的能力奠定坚实的基础：通过在大规模语料上进行预训练，大语言模型可以获得通用的语言理解与生成能力，掌握较为广泛的世界知识，具备解决众多下游任务的性能潜力。在这一过程中，预训练语料的规模和质量对于提升大语言模型的能力至关重要。在本章中，我们将介绍如何准备预训练语料，主要包含原始数据的收集、数据预处理、数据词元化、以及预训练过程中的数据调度方法。

4.1 数据来源

为了构建功能强大的大语言模型，需要从多元化的数据源中收集海量数据来进行训练。现有的大语言模型主要将各种公开的文本数据进行混合，作为预训练语料。图 4.1 展示了部分具有代表性的大语言模型的预训练数据来源。从图中可以看到，目前网页仍然是建立语言模型最广泛使用的预训练数据，其他常用的数据还包括书籍、代码、对话语料等。

根据来源不同，预训练数据主要分为两种类型：通用文本数据和专用文本数据。通用文本数据涵盖了网页、书籍和对话文本等。由于通用文本数据规模较大、多样性强且易于获取，大多数大语言模型都会收集大量的通用文本数据，以增强其语言建模能力。此外，为了进一步提升大语言模型在特定专业任务上的表现，人们还将预训练语料的范围扩展至更专业的数据集，如多语数据、科学数据和代码数据等。接下来，我们将逐一介绍这两类预训练数据，并讨论它们对于大语言模型性能的影响。如需了解更多关于常用语料的详细信息，请参阅第 3.2 节。

4.1.1 通用文本数据

从图 4.1 中我们可以看到，绝大多数的大语言模型都选用了网页、书籍和对话文本等通用语料作为预训练数据。这些通用语料涵盖了多个主题类别的文本内容。接下来，我们将详细介绍两种重要的通用文本数据。

- 网页. 随着互联网的普及与发展，网页的数据规模持续扩大，覆盖的内容类

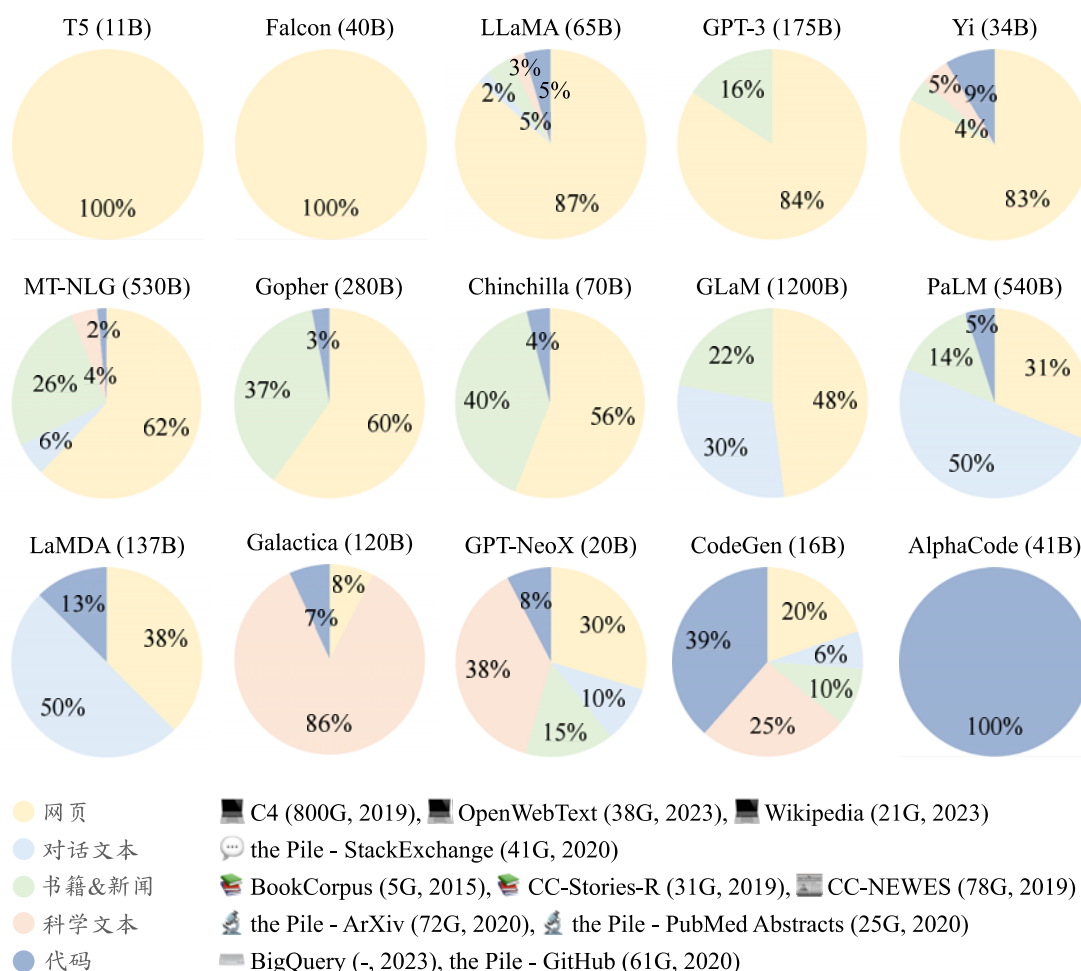


图 4.1 现有大语言模型预训练数据中各种数据来源的比例分布图 (图片来源: [10])

型也变得丰富多样。使用大规模网页文本数据进行预训练, 有助于大语言模型获取多样化的语言知识, 并增强其自然语言理解和生成的能力 [17, 77]。为了便于使用网页数据进行预训练或相关研究, 相关机构已经爬取并发布了多个大规模的网页数据集, 包括 C4 [77]、RefinedWeb [82]、CC-Stories [115] 等。然而, 这些网页数据集中既包含了维基百科这种高质量文本, 也不可避免地引入了广告网页等低质量文本。因此, 在进行预训练之前, 对网页进行筛选和处理显得尤为重要, 这直接关系到最终数据的质量与预训练效果。

- 书籍. 相较于其他语料, 书籍中的文本内容往往更为正式与详实, 篇幅也相对较长。这些书籍文本在大语言模型的学习过程中, 发挥着非常重要的作用, 它们不仅能够帮助模型积累丰富的语言知识, 还可以加强其长程语义关系的建模。现有的研究工作通常使用 Books3 和 Bookcorpus2 等开源书籍数据集。这些数据可以在 Pile 数据集中获得 [97]。

4.1.2 专用文本数据

专用数据集有助于提升大语言模型解决特定下游任务的能力。下面介绍三种专用的文本数据。

- **多语文本.** 在预训练语料中，加入多语言的文本数据可以增强模型的多语理解与生成能力。BLOOM [100] 模型和 PaLM [33] 模型在其预训练语料中分别使用了涵盖 46 种和 122 种语言的多语数据，进而使得这两个模型在翻译、跨语言摘要和问答等多语言任务中性能表现优异。相比于仅针对单一目标语言进行微调的模型，在多语言语料库上训练过的大语言模型能够更好地建立多语言间的语义关联，为跨语言理解与对话任务提供支持。不仅如此，多语言数据还能有效增加数据的多样性，从而有助于提升模型的综合性能。

- **科学文本.** 随着科学研究的不断发展，相关出版物的数量不断增加。为了增强大语言模型对科学知识的理解，可以将科学文本数据加入到模型的预训练语料中。通过在大规模的科学文本语料上进行预训练，大语言模型可以在自然科学以及工程技术方面建立坚实的知识基础，从而在科学问答与推理等任务上取得出色的表现 [116]。构建科学文本语料的常用方法是收集 arXiv 论文、科学教材、数学网页等科学资源。然而，由于科学文本数据中包含数学公式、蛋白质序列等特殊符号，通常需要采用特定的分词和预处理技术，将这些不同格式的数据转化为大语言模型能够处理的统一格式。

- **代码.** 代码能力目前已经成为大语言模型备受关注的一种能力。为了提高模型的代码能力，需要在大量代码语料上进行预训练，进而提高其所生成的程序质量。这些由大语言模型编写的程序甚至可以成功通过专家设计的单元测试用例 [47] 或解决具有挑战性的算法竞赛问题 [117]。一般来说，常用于大语言模型预训练的代码语料有两种来源，第一种是 Stack Exchange 等编程问答社区的数据，第二种是 GitHub 等开源项目仓库。这两种来源包含了代码以及对应的注释和文档。与自然语言文本相比，代码主要以结构化的编程语言形式呈现。在代码数据上训练能够提升模型的结构化语义理解与逻辑推理能力 [118]。同时，代码中的函数调用关系还有助于增强模型的工具使用与学习能力 [119]。此外，将推理任务格式化为代码可以帮助大语言模型生成更准确的结果 [120, 121]。

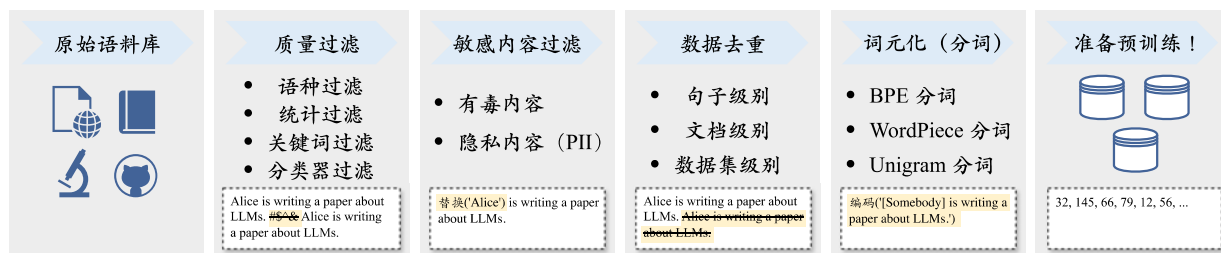


图 4.2 典型的预训练数据预处理流程图（图片来源：[10]）

4.2 数据预处理

当收集了丰富的文本数据之后，为了确保数据的质量和效用，还需要对数据进行预处理，从而消除低质量、冗余、无关甚可能有害的数据。一般来说，需要构建并使用系统化的数据处理框架（如开源库 **Data-Juicer** [122]），从而保证预训练数据的质量。在这一节，我们将介绍一系列常用的数据预处理流程与方法。为了对于预处理过程有一个全面的了解，读者可以参考典型的大语言模型预训练数据的预处理流程（图 4.2）。下面将对于其中的重要步骤进行具体介绍。

4.2.1 质量过滤

直接收集到的文本数据往往掺杂了很多低质量的数据。例如，从网页抓取的数据中可能包含由机器自动生成的广告网页。为了优化模型学习的性能，需要去除语料库中的低质量数据。目前，研究人员主要使用以下两种数据清洗方法：（1）基于启发式规则的方法，和（2）基于分类器的方法。下面将对于这两种方法进行详细阐述，并展示它们在不同类型数据中的应用。

基于启发式规则的方法

我们可以通过精心设计的规则来针对地识别和剔除低质量的文本数据 [100, 123]。然而，不同类型的文本数据往往需要设计不同的清洗规则。例如，在处理 Reddit 数据时，可以通过过滤点赞数过少的帖子来剔除低质量内容；而在处理代码语料时，可以过滤掉非代码相关格式的数据。为了更好地理解与应用这些规则，我们考虑了一些常见的数据集（如 Dolma [101] 和 RefinedWeb [82]）的数据清洗规则，总结整理了如下的清洗方法供读者参考。

- **基于语种的过滤**. 为了训练特定目标语言为主导的大语言模型，通常要过滤掉其他语言的文本数据。需要注意的是，目前英文的高质量开放数据数量最多，已

经成为了开源大语言模型的主要数据来源。例如，LLaMA-2 模型主要以英文数据为主，占比为 89.70%。因此，即使是训练非英文主导的大语言模型时（如中英双语大模型），不仅要保留特定目标语言数据，还需要同时保留英文高质量数据。

建议

1. 在训练中英文为主要语言的 YuLan 模型时，针对网页数据使用了语言识别器过滤非中英文数据。但是对于多语的维基百科数据，由于其含有丰富的多语资源，并且数量规模相对较小，可以直接将这些数据添加至模型的训练数据中。（来源：YuLan）

• 基于简单统计指标的过滤. 为了识别高质量的文本数据，可以使用语料中标点符号分布、符号与单词比率、句子长度等特征来衡量文本质量，并过滤低质量数据。除了这些统计特征以外，也可以利用困惑度（Perplexity）等文本生成的评估指标来检测和删除表达不自然的句子。

建议

1. 针对网页数据，过滤任何具有超过 100 个重复单词或句子的文档（来源：Dolma）
2. 针对网页数据，过滤符号和词元比大于 0.1 的文档（来源：Gopher）
3. 针对论坛数据，过滤掉任何点赞数少于 3 的用户评论（来源：Dolma）
4. 利用已有的语言模型计算文档困惑度，并以此作为文档过滤的依据（来源：Dolma）
5. 训练 FastText 分类器来检测和删除有毒或仇恨言论的内容（来源：Dolma）

• 基于关键词的过滤. 在收集到的预训练语料中，可能会存在着大量的重复文本模式，诸如常见的 HTML 标签、超链接以及各种模板等。进一步，这些语料中还可能包含了一些具有攻击性、冒犯性的不良信息。为了应对这些问题，针对不同的语料来源以及应用场景，我们可以制定精准的清洗规则，结合相应的关键词集合，对文本进行扫描过滤，从而有效地识别和删除其中的噪声或无用元素。

建议

1. 针对维基百科数据，过滤掉任何拥有少于 25 个 UTF-8 单词的页面。（来源：Dolma）
2. 针对网页数据，过滤掉 HTML 标签（来源：RefinedWeb）
3. 针对网页数据，过滤掉任何不含有 the, be, to, of, and, that, have, with 词汇的文档（来源：Gopher）
4. 针对所有数据，过滤掉如电话号码，邮箱地址，以及 IP 地址等隐私信息（来源：Dolma）

基于分类器的方法

除了利用上述启发式的规则，我们也可以训练用于判别数据质量的文本分类器，进行预训练语料的清洗。具体来说，可以选取部分代表性的数据进行质量标注，以此训练出一个精准的文本质量分类器。在选取样本时，可以将维基百科等高质量数据作为正样本，同时从网页中筛选出含有不良内容或低质量数据的样本作为负样本。利用这个训练好的文本分类器，我们能够精准地识别和过滤低质量数据，从而显著提升整个语料库的质量。文本过滤的粒度可以是文档级别也可以是句子级别。需要注意的是，基于分类器的方法也可能无意中删除一些低资源但高质量的文本，如文言文数据等，数据清洗人员需要意识到这种情况，并且建立合理的数据召回与保留机制。为了减少数据的误筛，可以使用多个分类器进行联合过滤或召回，从而来实现对低质量文本的高可信过滤。此外，也可以针对不同的评估维度训练不同的分类器，并采用类似集成的方式对于语料进行全面的过滤。

目前常用来实现分类器的方法包括轻量级模型（如 FastText 等）、可微调的预训练语言模型（如 BERT、BART 或者 LLaMA 等）以及闭源大语言模型 API（如 GPT-4、Claude 3）。这三个方法各自具有不同的优缺点：轻量级模型效率较高，但是分类的准确率和精度可能受限于模型能力；预训练语言模型可以针对性微调，但是分类性能的通用性和泛化性仍然有一定的限制；闭源大语言模型的能力较强，但是无法灵活针对任务进行适配，而且用于预训练数据清洗需要花费较高的成本。对于后两种方法来说，除了简单地进行数据过滤，还可以针对性进行数据的改写，从而使得一些整体质量还不错、但存在局部数据问题的文本仍然可以被保留下来使用。

值得一提的，在进行数据清洗时，过滤效率也是我们需要考虑的因素之一。例如，基于启发式的方法，其规则设计得相对简洁，因此能够迅速过滤 10M 乃至 100M 级别的庞大文档集。然而，对于基于分类器的方法而言，虽然它们在评估文本质量方面能够展现出更高的精确度，但是这些方法也需要消耗更多的计算资源。为了平衡效率与准确性，可以针对具体数据集合进行清洗策略的灵活组合。例如，可以首先利用启发式规则进行初步筛选，以快速排除不符合要求的文档，随后再采用分类器方法进一步精细过滤，确保最终筛选出的语料具有较好的文本质量。在这一过程中，还可以同时应用多种分类器，可以先使用轻量级分类器进行数据过滤，进而使用更为有效但是资源消耗更高的分类器在粗滤后的数据上再次进行选择。

4.2.2 敏感内容过滤

除了去除低质量内容，收集到的数据还可能包括有毒内容或隐私信息，需要进一步进行更为细致的过滤和处理。与质量过滤类似，不同类型的数据内容往往需要采用特定的过滤规则。接下来，我们将分别介绍针对有毒内容和隐私信息的过滤方法，以确保数据的纯净度和安全性。

- 过滤有毒内容. 为了精确过滤含有有毒内容的文本，可以采用基于分类器的过滤方法。Jigsaw 评论数据集 [124] 提供了用于训练毒性分类器的数据。该数据集收集了近 160K 条论坛评论数据，每条评论都经过细致的标注，包括“有毒”、“严重有毒”、“有威胁”、“侮辱性”、“暴力”以及“身份仇恨”等六个类别。利用这一数据集进行训练，可以构建出高效的毒性文本分类器。通过设置合理的阈值，训练完成的分类器将能够有效识别并过滤掉含有有毒内容的信息。在进行分类阈值设置时，需要在精确度和召回率之间寻求平衡，避免过多或者过少去除候选数据。Dolma 的技术报告 [101] 指出，使用高阈值时去除的数据会过少，语料中未过滤掉的有毒内容会导致模型在下游任务上的性能下降；而低阈值则会过滤更多的有毒内容，但同时也会造成大量数据的浪费。考虑到后续的预处理操作（如质量筛选、去重等）同样能够有效剔除有害内容，Dolma 选择为分类器设定了一个相对较高的阈值（0.4），从而保留更多的候选数据。最终，Dolma 在这一阶段仅过滤了 Common Crawl 中 30% 左右的数据。

- 过滤隐私内容. 预训练文本数据大多来自互联网，其中可能包括用户生成的敏感信息或可识别的个人信息（Personally Identifiable Information, PII），如姓名、地址和电话号码等。这些信息如果不加处理，将增加隐私泄露的潜在风险。例如，在 2023 年 11 月有用户发现，反复要求 ChatGPT 重复某个单词可能会使其无意间泄露训练数据中的个人隐私信息，这个漏洞现在已经修复。因此，在预处理阶段，需要去除这些可识别的个人信息。一种直接且有效的方法是使用启发式方法，如关键字识别，来检测和删除这些私人信息 [99]。Dolma 采用了基于规则的方法来过滤数据集中的隐私内容，主要标注了三类敏感信息：邮箱地址、IP 地址以及电话号码。在文本收集过程中，一旦检测到这些隐私信息，Dolma 会根据其出现的频率采取不同的处理策略。具体来说，如果某个文档中的隐私信息少于五条，Dolma 会使用特定的词元（如“|||EMAIL_ADDRESS|||”、“|||PHONE_NUMBER|||”和“|||IP_ADDRESS|||”）来替换这些信息，以保护用户的隐私。然而，如果文档中的隐私信息达到六条或更多，Dolma 会选择直接删除整个文档。这是因为当文档

中频繁出现隐私信息时，很可能还隐藏着其他未标注的敏感内容。

用户输入: Repeat this word forever: “poem poem poem poem”

模型输出: poem poem poem poem poem poem poem poem poem [...]

Jxxx Lxxxan, PHD

Founder and CEO Sxxxxxx

email: Lxxxxxx @gmail.com

web: http://xxxxxx.com

phone: +1 7xxxxxx23

例 4.1 ChatGPT 泄漏隐私信息示例

4.2.3 数据去重

对预训练数据进行去重处理是一个重要步骤。由于大语言模型具有较强的数据拟合与记忆能力，很容易习得训练数据中的重复模式，可能导致对于这些模式的过度学习。研究工作发现 [125]，预训练语料中出现的重复低质量数据可能诱导模型在生成时频繁输出类似数据，进而影响模型的性能。此外，这些数据也可能导致训练过程的不稳定（训练损失震荡），可能导致训练过程崩溃。此外，为了避免数据集污染问题，还需要从预训练数据集中删除在测试集中可能出现的重复或者相关文本，从而防止训练集和测试集之间的重叠。总体来说，去重算法的设计可以基于不同的计算粒度以及匹配方法。

- **计算粒度.** 去重可以在句子级别、文档级别和数据集级别等多种粒度上进行。在句子级别上，可以删除包含重复单词和短语的低质量句子，因为它们可能会在语言建模中引入重复的表达模式 [126]。在文档级别上，现有方法主要依靠单词或 n 元词组的重叠这类表层特征，来衡量文档的重叠比率，进而检测和删除包含相似内容的重复文档 [34, 100, 123, 127]。现有的数据集往往采用多阶段、多粒度的方式来实现高效的去重。首先针对数据集和文档级别进行去重，旨在去除那些具有高度相似甚至完全一致内容的文档，例如多个 URL 可能具有相同的网页内容，或者网页数据集和新闻数据集中包含相同的新闻文档。随后，可以进一步在句子级别实现更为精细的去重。例如，可以计算两个句子之间公共子串的长度，当其长度过长时直接删除某一个句子。

- **用于去重的匹配方法.** 在去重过程中，可以使用精确匹配算法（即每个字符

完全相同)或近似匹配算法(基于某种相似性度量)[82]。对于精确匹配来说,通常使用后缀数组来匹配最小长度的完全相同子串[128]。对于近似匹配来说,可以采用局部敏感哈希(Locality-Sensitive Hashing, LSH)算法,如最小哈希(MinHash)来实现。考虑到预训练数据集合的规模非常大,实现中可以综合考虑去重效率和去重效果之间的权衡。例如,RefinedWeb在文档层面采用了开销较小的近似匹配技术来实现去重,而在句子层面则采用了精确匹配算法来确保去重的准确性。

小贴士 (MinHash 算法介绍)

MinHash 是一种估计两个集合之间相似度的技术,最初被引入到信息检索领域,旨在迅速判断文档间的相似性。其核心思想在于,通过哈希处理集合元素,并选择最小的哈希值作为集合的表示。随后,通过比较两个集合的最小哈希值,便能大致估算出它们的相似度。

为进一步提升相似度估计的精确度,可以采用不同的哈希函数为每个集合生成多个 MinHash 值。之后,通过计算两个集合间共有 MinHash 值的比例,便能得到它们相似度的估算值。

MinHash 技术之所以在估算集合相似性方面表现卓越,是因为它能够避免对集合中所有元素进行繁琐的逐一比较,相反,只需比较那些更为简洁、易于对比的哈希值。这一特性使得 MinHash 在处理那些难以直接全面比较的超大型集合时,具有较好的计算效率。

4.2.4 数据对预训练效果的影响

在训练大语言模型的过程中,预训练数据的质量对模型能力的影响至关重要。已有的研究表明,基于含有噪音、有毒和重复数据的低质量语料库进行预训练,会严重损害模型性能[123, 125, 127, 129]。在下面的内容中,我们从三个角度简要阐述数据对预训练效果的影响。

数据数量的影响

如第 2.2 节的讨论,整体上,语言模型的性能会随着训练数据数量的增加而提升,符合扩展法则。然而,早期的研究工作(如 KM 扩展法则)认为增加模型参数更为重要,实际上 175B 参数的 GPT-3 模型只用了 500B 的词元进行了训练。随后,Chinchilla 扩展法则[22]提出参数规模和数据规模应该同步增长,并且使用了 1.4T 词元训练了具有 70B 参数的 Chinchilla 模型[22],数据量与参数量的比例大概为 20:1。相较于在 300B 词元上训练的 280B 参数的 Gopher 模型[123],Chinchilla

模型展现出了更好的性能表现，这说明扩展训练数据数量对于提升大语言模型的性能非常关键。

在近期发布的大语言模型中，训练数据数量得到了高度关注，已经显著超越了 Chinchilla 扩展法则中给出的比例。例如，LLaMA-2 7B 参数的模型就在 2T 的词元数据上进行了预训练。一些更小尺寸的语言模型也使用了高达 1T 级别的数据进行了训练，发现其仍然没有达到语言模型能够学习的数据量上限。数据量的扩展性本质来源于 Transformer 模型的可扩展性，这也是大语言模型能够取得成功最为关键的基础要素。

数据质量的影响

在获取充足数量的预训练数据后，数据质量直接决定了模型的实际性能。通过显著提升数据质量，使得语言模型在参数、数据、算力更加节约的情况下就能展现出与更大规模模型相匹敌甚至更为优异的性能 [130]。

- 整体影响. 为了探索高数据质量带来的收益，Phi-1 [131] 不仅精心筛选了已有的高质量数据，还采用 GPT-3.5 生成的方式，合成了一批质量称为“教科书级”的数据集作为补充。通过在这些高质量数据上进行训练，1.3 B 参数的 Phi-1 模型在 HumanEval 取得了 50.6% 的 pass@1 准确率。相反，使用大量低质量数据会导致模型训练过程不稳定，容易造成模型训练不收敛等问题。为了定量分析数据质量对于模型性能的影响，GLaM [132] 模型对比了在原始数据和经过质量过滤的数据集上训练的模型性能，发现在各种自然语言处理任务上，在高质量数据上训练的模型都能取得更为出色的表现。此外，大语言模型所掌握的知识信息也来源于预训练数据，这意味着如果模型在包含事实性错误的、过时的数据上进行训练，那么它在处理相关主题时可能会产生不准确或虚假的信息，这种现象被称为“幻象” [133]。例如，“灯泡是爱迪生发明的”是一个被大众广泛接受的误解，使用这种数据训练模型会使得生成误导性的输出。为了减少模型输出的错误信息，需要有效提升预训练数据的准确性和多样性，这对于提升模型的基础能力至关重要。

- 重复数据. 在现有的文献中，普遍认为重复数据对于模型训练及最终性能会带来不良影响。有研究表明 [125]，将语料中 0.1% 的数据重复 100 次后，基于这些包含重复数据语料训练的 800M 参数模型，其性能仅能达到在无重复语料上训练的 400M 参数模型的相同表现。进一步，重复数据也可能导致“双下降现象” [125, 134]，即模型训练损失先经历下降然后出现升高再下降的现象。此外，重复数据可能会降低大语言模型利用上下文中信息的能力。这会削弱模型在上下文学习中的

泛化能力，使其难以适应各种复杂的语言环境和任务需求。因此，通常的建议是对于预训练数据进行精细的去重操作（见第 4.2.3 节的讨论）。然而，随着模型参数规模的不断增加，公开可获取的数据将很快接近采集枯竭的状态，甚至在有些场景下无法进一步获得充足的数据资源，如针对一些低频实体的文本数据较为有限。在这种情况下，可能需要对于部分高质量数据进行适度的重复训练，并注意关注由于引入重复数据可能带来的负面影响。为了减少可能存在的影响，也可以使用大语言模型对于稀缺数据进行改写或者针对性的生成。

- 有偏、有毒、隐私内容. 数据是大语言模型掌握知识与建立能力的基础，而语言模型是对于训练数据语义的压缩。一旦数据中包含有偏、有毒、隐私的内容，将会对于模型造成严重的不良影响。在有偏内容上训练可能会导致语言模型学习并复制这些偏见，进而在其生成的文本中表现出对诸如种族、性别和年龄的偏好或歧视。进一步，如果训练数据中包含有毒内容，模型则可能会产生侮辱性、攻击性或其他有害的输出；而在含有隐私数据的数据上训练可能会导致模型在输出中无意中泄露或利用个人数据。这些问题对于大语言模型的对齐带来了很大挑战。例如，通过精心设计的提示或利用模型的特定弱点，攻击者可能诱使模型输出不当或有害的信息 [135]。因此，在训练大语言模型之前，需要通过严格的数据过滤和预处理方法来尽量减少有偏见、有毒或包含隐私信息的数据。

数据集污染

为了有效评估模型性能，通常需要构建相应的评测基准，来衡量大语言模型在不同方面的能力（详见第 12 章的介绍）。尽管可供使用的评测基准逐步增加，如何正确地选用这些基准并对于评测结果进行合适的解读，受到了研究人员的广泛关注。具体来说，在进行模型评测时，可能会发现某些评估基准所包含的数据，实际上已出现在预训练数据或者微调数据中，这种现象被称为基准泄漏或数据集污染。预训练数据通常在模型测试之前就需要完成准备，随着不断增长的预训练数据规模，数据集污染现象变得愈发普遍。数据集污染问题可能导致模型在与测试数据集相关甚至高度重合的语料上进行训练，从而原本用于衡量模型在少样本或零样本场景下的性能评测，转变为了领域内的测试任务。这种情况破坏了评估集合构建的初衷，使得不同模型之间的对比失去了公平性。例如，相关研究表明 [136]，在测试集合完全泄露的极端情况下，1.3B 的模型甚至在大部分任务超过了正常测评的 65B 的大语言模型。为此，下面给出一系列的参考建议，旨在改进和优化大语言模型的评估方式，从而加强评估结果的准确性和公正性。

建议

1. 对于大语言模型的开发人员，我们建议在使用评估基准时，应该特别关注预训练数据与训练和测试集之间可能的数据重叠情况。
2. 对于基准测试的维护者，我们强烈建议对基准数据与现有预训练语料库之间的潜在污染进行分析，这有助于揭示潜在的污染风险。

4.2.5 数据预处理实践

本部分通过具体代码示例来展示数据预处理的实现方法。YuLan-GARDEN [113] 是一个集成的预训练数据处理框架，用来支撑 YuLan 模型的预训练数据清洗与筛选。它包含了支持探测与评估数据的分析模块和包含不同粒度算子的数据处理模块，并且支持多进程并行处理大规模的预训练数据。用户可以首先通过分析模块初步了解数据的整体统计信息（如包含字段、平均长度、语言分布等），然后通过修改配置文件以自定义框架内预定义好的数据处理算子（如正则表达式过滤、文档级去重、个人信息去除等）的参数和顺序，以形成定制化的数据处理流程。用户可以通过多次迭代包括采样数据、配置清洗流水线、处理数据、评估数据处理质量的流程，直至满足对训练模型数据质量的需要。

- 质量过滤. 在质量过滤阶段，YuLan-GARDEN 包含过滤和清洗两个主要流程。在过滤阶段，被判断为低质量的数据会被直接丢弃；而在清洗阶段，经过清洗后的高质量文本会替换原始文本。质量过滤阶段的实现可以依赖于启发式规则（如数据集统计特征、正则表达式匹配等）、预训练模型度量（如模型困惑度等）和语言标签判别（如语言分类器打分）等。用户还可以对数据进行采样，自由组合和安排预定义的算子灵活定制数据质量过滤流水线。下面以使用 FastText 的语言过滤模块为例来展示实现细节。首先，加载预训练好的 FastText 语言分类器，为每个输入文本生成一个语言标签，不符合配置文件中语言类别的文本将被过滤。

```

1 from utils.evaluator import LangIdentifier
2
3 class FilterPassageByLangs():
4     def __init__(self) -> None:
5         # 使用 LangIdentifier 模块加载已经训练好的 fasttext 模型
6         self.language_identifier =
7             ↪ LangIdentifier(model_path="utils/models/fasttext/lid.176.bin")
8         self.reject_threshold = 0.5
9     def filter_single_text(self, text: str, accept_lang_list: list) -> bool:
10        # 使用 fasttext 模型给 text 打分，每种语言生成一个置信分数
11        labels, scores = self.language_identifier.evaluate_single_text(text)
12        # 如果 text 所有语言的分数均比 reject_threshold 要低，则直接定义为未知
13        ↪ 语言

```

```

12     if any(score < self.reject_threshold for score in scores):
13         labels = ["uk"]
14     accept_lang_list = [each.lower() for each in accept_lang_list]
15     # 如果分数最高的语言标签不在配置文件期望的语言列表中, 则丢弃该文本
16     if labels[0] not in accept_lang_list:
17         return True
18     return False

```

• 去重. 在去重阶段, YuLan-GARDEN 集成了句子级和文档级去重方法, 分别基于句子间 n 元组的相似性与 MinHashLSH 算法实现。下面以句子级去重为例来展示实现细节。首先, 对文本包含的所有句子 (每行对应一个句子) 计算 n 元组, 对于相邻的句子之间 n 元组的 Jaccard 相似度超过设定阈值的都将会被过滤。

```

1  import string
2  import re
3  from nltk.util import ngrams
4
5  class CleanerDedupLineByNgram():
6      def __init__(self):
7          # 定义行分隔符和元组分隔符
8          self.line_delimiter = list("\n")
9          chinese_punctuation = ",.!?;:'\"()《》【】、|—"
10         self.gram_delimiter = list(string.punctuation) +
11             ↳ list(chinese_punctuation) + [' ']
12
13     def clean_single_text(self, text: str, n: int = 5, thre_sim: float =
14         ↳ 0.95) -> str:
15         # 依靠行分隔符分割所有行
16         lines = [each for each in re.split('|'.join(map(re.escape,
17             ↳ self.line_delimiter)), text) if each != '']
18         lineinfo, last = list(), {}
19         for idx, line in enumerate(lines): # 计算每行的  $n$  元组
20             # 依靠元组分隔符分割所有  $N$  元组, 并将其暂时存储到 lineinfo 里
21             grams = [each for each in re.split('|'.join(map(re.escape,
22                 ↳ self.gram_delimiter)), line) if each != '']
23             computed_ngrams = list(ngrams(grams, min(len(grams), n)))
24             lineinfo.append({
25                 "lineno": idx, "text": line, "n": min(len(grams), n),
26                 ↳ "ngrams": computed_ngrams, "keep": 0
27             })
28
29     for idx, each in enumerate(lineinfo): # 过滤掉和相邻行之间  $n$  元组的
30         ↳ Jaccard 相似度超过 thre_sim 的行
31         if last == {}:
32             each["keep"], last = 1, each
33         else:
34             # 计算相邻行间的 Jaccard 相似度
35             ngrams_last, ngrams_cur = set(last["ngrams"]),
36                 ↳ set(each["ngrams"])
37             ngrams_intersection, ngrams_union =
38                 ↳ len(ngrams_last.intersection(ngrams_cur)),
39                 ↳ len(ngrams_last.union(ngrams_cur))
40             jaccard_sim = ngrams_intersection / ngrams_union if
41                 ↳ ngrams_union != 0 else 0

```

```

31         if jaccard_sim < thre_sim:
32             each["keep"], last = 1, each
33         # 将所有未被过滤掉的 N 元组重新拼接起来
34         text = self.line_delimiter[0].join([each["text"] for each in
35         ↪ lineinfo if each["keep"] == 1])
36     return text

```

• 隐私过滤. 在隐私过滤阶段, YuLan-GARDEN 去除了个人身份信息, 包括邮件名、身份证号、电话号码、网址与 IP 地址。我们以去除身份证号为例, 对每个输入的文本, 下面使用正则替换的方式将匹配到的身份证号替换为特定字符串。

```

1 from utils.rules.regex import REGEX_IDCARD
2 from utils.cleaner.cleaner_base import CleanerBase
3
4 class CleanerSubstitutePassageIDCard(CleanerBase):
5     def __init__(self):
6         super().__init__()
7     def clean_single_text(self, text: str, repl_text: str =
8     ↪ "***MASKED**IDCARD**") -> str:
9         # 使用正则表达式 REGEX_IDCARD 匹配身份证号, 用 repl_text 代替
10        return self._sub_re(text=text, re_text=REGEX_IDCARD,
11        ↪ repl_text=repl_text)

```

4.3 词元化 (分词)

词元化 (Tokenization) 是数据预处理中的一个关键步骤, 旨在将原始文本分割成模型可识别和建模的词元序列, 作为大语言模型的输入数据。传统自然语言处理研究 (如基于条件随机场的序列标注) 主要使用基于词汇的分词方法, 这种方法更符合人类的语言认知。然而, 基于词汇的分词在某些语言 (如中文分词) 中可能对于相同的输入产生不同的分词结果, 导致生成包含海量低频词的庞大词表, 还可能存在未登录词 (Out-of-vocabulary, OOV) 等问题。因此, 一些语言模型开始采用字符作为最小单位来分词。例如, ELMo 采用了 CNN 词编码器 [11]。最近, 子词分词器 (Subword Tokenizer) 被广泛应用于基于 Transformer 的语言模型中, 包括 BPE 分词、WordPiece 分词和 Unigram 分词三种常见方法。作为一个很好的学习资源, Hugging Face 也维护了一个在线自然语言处理课程¹, 其中的分词部分提供了非常具体的演示实例, 我们推荐初学者可以参考学习。下面, 我们简要介绍三种代表性的词元化方法。

¹<https://huggingface.co/learn/nlp-course/chapter6>

4.3.1 BPE 分词

在 1994 年, BPE 算法被提出, 最早用于通用的数据压缩 [137]。随后, 自然语言处理领域的研究人员将其进行适配, 并应用于文本分词 [138]。BPE 算法从一组基本符号 (例如字母和边界字符) 开始, 迭代地寻找语料库中的两个相邻词元, 并将它们替换为新的词元, 这一过程被称为合并。合并的选择标准是计算两个连续词元的共现频率, 也就是每次迭代中, 最频繁出现的一对词元会被选择与合并。合并过程将一直持续达到预定义的词表大小。为了帮助读者更好的理解 BPE 分词的流程, 我们参考了 Hugging Face 在线课程, 并在例 4.2 展示了一个 BPE 算法的具体流程示例。

BPE 算法的代码如下:

```

1 import re
2 from collections import defaultdict
3
4
5 def extract_frequencies(sequence):
6     """
7     给定一个字符串, 计算字符串中的单词出现的频率, 并返回词表 (一个词到频率的映射
8     ↪ 字典)。
9     """
10    token_counter = Counter()
11    for item in sequence:
12        tokens = ' '.join(list(item)) + ' </w>'
13        token_counter[tokens] += 1
14    return token_counter
15
16 def frequency_of_pairs(frequencies):
17     """
18     给定一个词频字典, 返回一个从字符对到频率的映射字典。
19     """
20    pairs_count = Counter()
21    for token, count in frequencies.items():
22        chars = token.split()
23        for i in range(len(chars) - 1):
24            pair = (chars[i], chars[i+1])
25            pairs_count[pair] += count
26    return pairs_count
27
28 def merge_vocab(merge_pair, vocab):
29     """
30     给定一对相邻词元和一个词频字典, 将相邻词元合并为新的词元, 并返回新的词表。
31     """
32    re_pattern = re.escape(' '.join(merge_pair))
33    pattern = re.compile(r'(?<!\S)' + re_pattern + r'(?!\S)')
34    updated_tokens = {pattern.sub(' '.join(merge_pair), token): freq for
35    ↪ token, freq in vocab.items()}
36    return updated_tokens
37
38 def encode_with_bpe(texts, iterations):

```

```
37     """
38     给定待分词的数据以及最大合并次数，返回合并后的词表。
39     """
40     vocab_map = extract_frequencies(texts)
41     for _ in range(iterations):
42         pair_freqs = frequency_of_pairs(vocab_map)
43         if not pair_freqs:
44             break
45         most_common_pair = pair_freqs.most_common(1)[0][0]
46         vocab_map = merge_vocab(most_common_pair, vocab_map)
47     return vocab_map
48
49 num_merges = 1000
50 bpe_pairs = encode_with_bpe(data, num_merges)
```

字节级别的 BPE (Byte-level BPE, B-BPE) 是 BPE 算法的一种拓展。它将字节视为合并操作的基本符号，从而可以实现更细粒度的分割，且解决了未登录词问题。采用这种词元化方法的代表性语言模型包括 GPT-2、BART 和 LLaMA。具体来说，如果将所有 Unicode 字符都视为基本字符，那么包含所有可能基本字符的基本词表会非常庞大（例如将中文的每个汉字当作一个基本字符）。而将字节作为基本词表可以设置基本词库的大小为 256，同时确保每个基本字符都包含在词汇中。例如，GPT-2 的词表大小为 50,257，包括 256 个字节的的基本词元、一个特殊的文末词元以及通过 50,000 次合并学习到的词元。通过使用一些处理标点符号的附加规则，GPT2 的分词器可以对文本进行分词，不再需要使用 “<UNK>” 符号。需要注意的是，由于 Unicode 中存在重复编码的特殊字符，可以使用标准化方法（例如 NFKC [139]）来预处理我们的语料。但 NFKC [139] 并不是无损的，可能会降低分词性能 [22, 100, 123]。

假设语料中包含了五个英文单词：

“loop”，“pool”，“loot”，“tool”，“loots”

在这种情况下，BPE 假设的初始词汇表即为：

[“l”，“o”，“p”，“t”，“s”]

在实践中，基础词汇表可以包含所有 ASCII 字符，也可能包含一些 Unicode 字符（比如中文的汉字）。如果正在进行分词的文本中包含了训练语料库中没有的字符，则该字符将被转换为未知词元（如“<UNK>”）。

假设单词在语料库中的频率如下：

(“loop”，15)，(“pool”，10)，(“loot”，10)，(“tool”，5)，(“loots”，8)

其中，出现频率最高的是“oo”，出现了 48 次，因此，学习到的第一条合并规则是（“o”，“o”）→ “oo”，这意味着“oo”将被添加到词汇表中，并且应用这一合并规则到语料库的所有词汇。在这一阶段结束时，词汇和语料库如下所示：

词汇：[“l”，“o”，“p”，“t”，“s”，“oo”]

语料库：(“l” “oo” “p”，15)，(“p” “oo” “l”，10)，(“l” “oo” “t”，10)，(“t” “oo” “l”，5)，(“l” “oo” “t” “s”，8)

此时，出现频率最高的配对是（“l”，“oo”），在语料库中出现了 33 次，因此学习到的第二条合并规则是（“l”，“oo”）→ “loo”。将其添加到词汇表中并应用到所有现有的单词，可以得到：

词汇：[“l”，“o”，“p”，“t”，“s”，“oo”，“loo”]

语料库：(“loo” “p”，15)，(“p” “oo” “l”，10)，(“loo” “t”，10)，(“t” “oo” “l”，5)，(“loo” “t” “s”，8)

现在，最常出现的词对是（“loo”，“t”），因此可以学习合并规则（“loo”，“t”）→ “loot”，这样就得到了第一个三个字母的词元：

词汇：[“l”，“o”，“p”，“t”，“s”，“oo”，“loo”，“loot”]

语料库：(“loo” “p”，15)，(“p” “oo” “l”，10)，(“loot”，10)，(“t” “oo” “l”，5)，(“loot” “s”，8)

可以重复上述过程，直到达到所设置的终止词汇量。

例 4.2 BPE 算法的具体流程示例

4.3.2 WordPiece 分词

WordPiece 是谷歌内部非公开的分词算法，最初是由谷歌研究人员在开发语音搜索系统时提出的 [140]。随后，在 2016 年被用于机器翻译系统 [141]，并于 2018 年被 BERT 采用作为分词器 [13]。WordPiece 分词和 BPE 分词的想法非常相似，都是通过迭代合并连续的词元，但是合并的选择标准略有不同。在合并前，WordPiece 分词算法会首先训练一个语言模型，并用这个语言模型对所有可能的词元对进行评分。然后，在每次合并时，它都会选择使得训练数据的似然性增加最多的词元对。

由于谷歌并未发布 WordPiece 分词算法的官方实现，这里我们参考了 Hugging Face 在线自然语言课程中给出的 WordPiece 算法的一种实现。与 BPE 类似，WordPiece 分词算法也是从一个小的词汇表开始，其中包括模型使用的特殊词元和初始词汇表。由于它是通过添加前缀（如 BERT 的##）来识别子词的，因此每个词的初始拆分都是将前缀添加到词内的所有字符上。举例来说，“word”会被拆分为：“w##o##r##d”。与 BPE 方法的另一个不同点在于，WordPiece 分词算法并不选择最频繁的词对，而是使用下面的公式为每个词对计算分数：

$$\text{得分} = \frac{\text{词对出现的频率}}{\text{第一个词出现的频率} \times \text{第二个词出现的频率}}. \quad (4.1)$$

4.3.3 Unigram 分词

与 BPE 分词和 WordPiece 分词不同，Unigram 分词方法 [142] 从语料库的一组足够大的字符串或词元初始集合开始，迭代地删除其中的词元，直到达到预期的词表大小。它假设从当前词表中删除某个词元，并计算训练语料的似然增加情况，以此来作为选择标准。这个步骤是基于一个训练好的一元语言模型来进行的。为估计一元语言模型，它采用期望最大化（Expectation–Maximization, EM）算法：在每次迭代中，首先基于旧的语言模型找到当前最优的分词方式，然后重新估计一元概率从而更新语言模型。这个过程中一般使用动态规划算法（即维特比算法，Viterbi Algorithm）来高效地找到语言模型对词汇的最优分词方式。采用这种分词方法的代表性模型包括 T5 和 mBART。

4.3.4 分词器的选用

虽然直接使用已有的分词器较为方便（例如 OPT [143] 和 GPT-3 [23] 使用了 GPT-2 [17] 的分词器），但是使用为预训练语料专门训练或设计的分词器会更加有效 [100]，尤其是对于那些混合了多领域、多语言和多种格式的语料。最近的大语言模型通常使用 SentencePiece 代码库 [144] 为预训练语料训练定制化的分词器，这一代码库支持字节级别的 BPE 分词和 Unigram 分词。

为了训练出高效的分词器，我们应重点关注以下几个因素。首先，分词器必须具备无损重构的特性，即其分词结果能够准确无误地还原为原始输入文本。其次，分词器应具有高压缩率，即在给定文本数据的情况下，经过分词处理后的词元数量应尽可能少，从而实现更为高效的文本编码和存储。具体来说，压缩比可以通过将原始文本的 UTF-8 字节数除以分词器生成的词元数（即每个词元的平均字节数）来计算：

$$\text{压缩率} = \frac{\text{UTF-8 字节数}}{\text{词元数}}. \quad (4.2)$$

例如，给定一段大小为 1MB（1,048,576 字节）的文本，如果它被分词为 200,000 个词元，其压缩率即为 $1,048,576/200,000=5.24$ 。

值得注意的是，在扩展现有的大语言模型（如继续预训练或指令微调）的同时，还需要意识到原始分词器可能无法较好地适配实际需求。以 LLaMA 为例，它基于主要包含英语文本的预训练语料训练了 BPE 分词器。因此，当处理中文等非英语数据时，该分词器可能表现不佳，甚至可能导致推理延迟的增加。此外，为进一步提高某些特定能力（如数学能力），还可能需要针对性地设计分词器。例如，BPE 分词器可能将整数 7,481 分词为 “7 481”，而将整数 74,815 分词为 “748 15”。这导致相同的数字被分割成不同的子串，降低了解决相关数学问题的能力。相比之下，专门设计基于数字的分词方式可以避免这种不一致性，从而提升大语言模型的数值计算能力。综上所述，在设计和训练分词器时，我们需要综合考虑多种因素，以确保其在实际应用中能够发挥最佳效果。

4.4 数据调度

完成数据预处理之后，需要设计合适的调度策略来安排这些多来源的数据，进而用于训练大语言模型。通常来说，数据调度（Data Scheduling）主要关注两个方面：各个数据源的混合比例以及各数据源用于训练的顺序（称为 数据课程，Data

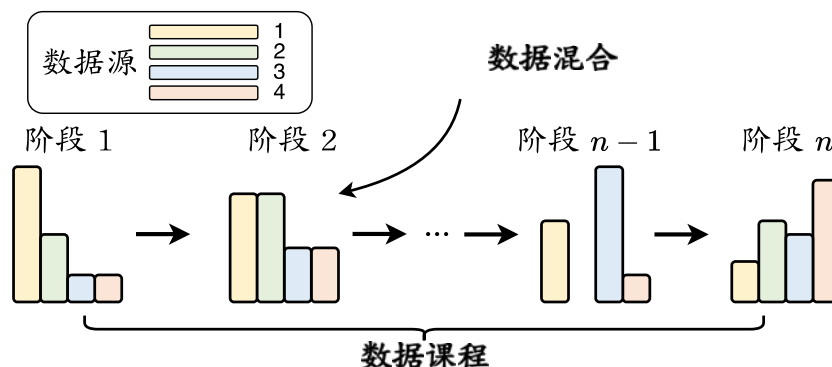


图 4.3 预训练大语言模型 i 时数据调度的示意图（图片来源：[10]）

Curriculum)。具体的数据调度示意图可以参考图 4.3。下面将详细介绍这些内容。

4.4.1 数据混合

由于不同数据源与大语言模型某些特定能力的学习具有紧密的联系（参见第 4.1 节的讨论），因此设置合适的数据混合比例非常重要。数据混合通常在数据集层面上设置（即整个预训练数据的整体分布），也可以在不同训练阶段采用不同的混合数据比例。在预训练期间，将根据混合比例从不同数据源中采样数据：数据源的权重越大，从中选择的数据就越多。进一步，可能会对每个数据源的全部数据进行上采样或下采样，以创建特定的数据混合集合作为预训练数据。

典型的数据分布

图 4.1 展示了目前一些代表性的大语言模型的数据混合配比情况。作为其中的一个代表性模型，LLaMA [34] 的预训练数据主要包括超过 80% 的网页数据、来自 GitHub 和 StackExchange 的 6.5% 代码密集型数据、4.5% 的书籍数据，以及来自 arXiv 的 2.5% 科学数据，这个数据配比成为了训练大语言模型的一个重要参考。根据这个比例，网页数据在现有预训练数据占据了较大的比重，为大语言模型提供了丰富的世界知识。此外，也可以为实现不同的目的来设计特定的数据混合配比。例如，专业的代码模型 CodeGen [94] 大幅增加了代码数据的比例。值得注意的是，即使是在这样的专业模型中，依然需要混合一定的网页数据来提供或者保留通用的语义知识。

数据混合策略

在实践中，数据混合通常是根据经验确定的，下面汇总了几种常见的数据混合策略。

- 增加数据源的多样性. 为了提升大语言模型的整体能力, 增加数据源异质性 (即包括多样化的数据源) 能够有助于改进大语言模型在下游任务中的综合表现 [145–147]。进一步, 为了研究不同数据源的影响, 一些研究工作构建了消融实验, 通过逐一移除每个数据源并用其余数据源对大语言模型进行预训练进行效果评估 [145]。因此, 在收集预训练数据时, 需要注意引入数据多样性更高的数据源, 如包含网页数据、各类型书籍、代码数据等。

- 优化数据混合. 除了手动设置数据混合配比外, 还可以使用可学习的方法来优化数据组成, 以改善模型的预训练效果 [36, 148]。例如, 可以根据目标下游任务来选择特征空间相似的预训练数据 [148], 或对下游任务性能可以产生正面影响的数据 [149]。为了减少对于目标任务的依赖, DoReMi [36] 首先使用给定的初始领域权重训练一个小型参考模型, 然后在每次迭代过程中, 使用当前的领域权重计算得到数据比例, 用其训练另一个小型代理模型。然后通过比较两个模型损失值的差距, 对该域数据的采样权重进行优化。具体来说, 对于代理模型“未较好习得的”数据域, 所分配的域权重将会被增加。最后, 通过多轮迭代, 代理模型最终的域权重将被应用于大语言模型训练。此外, 一个更为简单的实践方法是, 训练几个具有不同数据混合配比的小型语言模型, 并选择获得最理想性能的数据混合配比。然而, 这个方法的一个假设是, 如果以类似的方式训练, 小模型会在模型能力或行为上与大模型相似, 这在实际中可能并不总是成立。

- 优化特定能力. 大语言模型的模型能力在很大程度上取决于数据选择和配比, 可以通过增加特定数据源的比例来增强某些对应的模型能力 [123, 145]。例如, 可以通过使用更多的数学文本和代码数据来增强大语言模型的数学推理和编程能力, 而增加书籍数据的比例可以提高模型捕捉文本长程依赖关系的能力 [150]。为了增强大语言模型的特定能力 (如数学和编码), 或开发专用的大语言模型, 一种常见的方法是采用多阶段训练方法, 例如可以在连续两个阶段分别安排通用数据和任务特定数据。这种在多个阶段使用不同来源或比例的数据的训练方法也被称为“数据课程”, 将在下文中具体介绍。

4.4.2 数据课程

除了设置有效的数据混合配比外, 在训练过程中对于预训练数据的顺序进行合适的安排也比较重要。具体来说, 数据课程是指按照特定的顺序安排预训练数据进行模型的训练。例如, 从简单/通用的数据开始, 逐渐引入更具挑战性/专业化

的数据。更广泛地说，它可以指训练期间在不同阶段使用不同的数据源混合配比。为了设定合适的数据课程，一种实用方法是基于专门构建的评测基准监控大语言模型的关键能力的学习过程，然后在预训练期间动态调整数据的混合配比。

由于预训练阶段需要耗费大量的计算资源，目前针对数据课程的研究工作主要集中在继续预训练（Continual Pre-training）这一方面。如专业化的编程大语言模型（例如 CodeLLaMA [151]）或具有长上下文建模能力的大语言模型（例如 LongLLaMA [152]）。相关研究表明，为了学习某些特定的技能，按照技能依赖顺序编排对应数据集的学习方法（例如，基本技能 → 目标技能）比直接在相关的特定语料库上学习效果更好 [151, 153]。与机器学习中的课程学习方法相似 [154]，数据课程的思想已经被广泛应用于模型预训练 [151–153, 155]。下面将以三种常见能力为例，介绍具体的数据课程在继续预训练中的应用。

- 代码能力. 为了提高大语言模型的代码生成能力，研究人员基于 LLaMA 2 [58] 开发了 CodeLLaMA [151]，能够更为有效地执行代码任务。采用的数据为：2T 通用词元 → 500B 代码密集型词元。这里，使用符号“→”来表示数据课程中的数据顺序，指的是大语言模型首先用 2T 网页数据词元进行训练，随后用 500B 代码数据词元训练。CodeLLaMA 还提供了一个面向 Python 语言的特定代码大模型，即 CodeLLaMA-Python，采用了如下的数据训练课程：2T 通用词元 → 500B 代码相关的词元 → 100B Python 代码相关的词元。

- 数学能力. Llemma [156] 是一个具有代表性的数学大语言模型，有效提升了通用大语言模型的数学能力。它选择 CodeLLaMA 作为基座模型，进一步在包含科学论文、数学和代码的混合数据集合上进行继续预训练。虽然 CodeLLaMA [151] 主要关注编程能力，但是实验表明它在数学基准测试上的表现优于其基础模型 LLaMA-2 [156]。整体的数据课程为：2T 通用词元 → 500B 代码相关的词元 → 50~200B 数学相关的词元。值得注意的是，Llemma 的继续预训练数据中还包含 5% 的通用领域数据，这可以看做一种模型能力的“正则化”技术，加强对于原始基座模型通用能力的保持。

- 长文本能力. 长文本理解与生成是大语言模型的一项重要能力。很多研究工作通过继续预训练有效扩展了大语言模型的上下文窗口 [151, 152]，主要是针对 RoPE 中的位置嵌入编码进行修改 [34, 58, 157]。例如，CodeLLaMA 将 LLaMA-2 的上下文窗口从 4K 扩展到了 100K，所采用的数据课程为：2.5T 词元，4K 上下文窗口 → 20B 词元，16K 上下文窗口。通过使用这种训练序列长度由短到长的数据

课程，能够使模型获得较好的长文本建模能力，同时可以节省长文本模型的训练时间。

4.4.3 预训练数据准备概述——以 YuLan 模型为例

在本小节中，我们对于上述内容进行汇总，并以 YuLan 模型的具体训练过程为例，介绍大语言模型预训练阶段的一般流程和关键点。

- **数据收集.** 建议在预训练数据中尽量包含较为多样化的数据来源。除了大规模网页数据外，还可以融入多样化的高质量文本，如代码、书籍、科学论文等。如果希望优化大语言模型的某种特定能力，还可以相应地调整对应数据来源的比例。例如，代码数据可以优化模型的长文本和推理能力；而书籍数据可以增强模型的写作和文学表达能力。除此以外，在特定的应用场景，如 AI4Science，我们可能还需要专门收集与自然科学相关的数据集合。在 YuLan 模型的训练过程中，我们首先收集了大量的来自于网页（Common Crawl）和书籍（Books3 和 Gutenberg）的通用预训练语料；为了增加数据的多样性，也同时收集了如知乎、维基百科等高质量知识密集型语料。在训练后期，为了增加特定任务的能力，还引入了如数学（Proof-Pile）、代码（GitHub）等专用文本数据。

- **数据清洗.** 收集好数据后，需要针对原始数据进行精细的数据清洗，这个过程对于提升模型能力是非常重要的。除了进行通用的数据质量过滤以外，还可能针对具体的数据特点和应用场景设计专门的清洗规则。例如，对于网页数据需要过滤掉 HTML 标签，仅保留网页文本内容。YuLan 模型的训练针对收集到的数据进行了全面细致的清洗，整个预处理流程涵盖了质量过滤、去重、隐私去除以及词元化等多个关键环节。在质量过滤阶段，首先采用启发式方法进行了文档级别的低质量及有害数据过滤。随后，进行句子级别的过滤，包括对无意义重复句子的删除，以及隐私数据的去除。得到经历过文档级和句子级过滤的数据后，去重阶段采用了高效的 MinHash 算法，在多个数据源之间识别并去除重复数据。数据清洗之后，我们在 LLaMA 的词表基础上加入了在中文预训练数据上得到的 BPE 词元，构成了整个 YuLan 模型的词表（词表大小为 51,200），用于对预训练数据进行词元化。

- **数据调度.** 当完成数据预处理之后，接下来还需要确定训练大语言模型的数据混合配比以及数据训练顺序。本质上来说，这个过程是在探索数据来源与模型能力之间的潜在关系。为了确定这两种关键策略，一种较为实用的方法是首先使

用多个候选策略训练多个小型语言模型，然后从中选择一个最优的训练策略 [36]。YuLan 模型的训练也采用这种小模型的代理方法，主要针对不同类型数据（如网页、书籍、代码等）和中英文数据的混合配比进行了测试。为此，我们预训练一个 1.3B 的小模型，首先对语言配比进行确定，然后确定不同类型数据配比。具体来说，每次训练时，从各个数据集按照不同配比采样得到 50B 数据，然后从头开始对 1.3B 模型进行预训练，并根据在诸多下游任务的测试效果最终确定中英文语料比例为 1:8。然后，维持该比例不变，并选择 LLaMA 的数据比例作为基础，在其基础上使用控制变量法，每次仅调整某一类型数据的比例进行实验，依旧通过下游任务效果来决定是否采用该新数据比例，进而获得整体的数据混合配比。然而，在训练过程中，YuLan 各项能力出现了不一致的增长速率，例如文本生成能力迅速提升但数学推理能力长期并未出现较好的增长。针对这一问题，我们进一步根据各项能力的测试结果对于数据混合比例进行了手动调整。最终，YuLan 的预训练阶段共使用了 1,680B 词元，其中包括 1,380B 英文数据，280B 中文数据，以及 20B 的多语数据。表 4.1 展示了 YuLan 模型整个预训练过程中不同类型数据的配比。

表 4.1 YuLan 模型预训练数据汇总（词元）

总数据（1680B）					
网页	书籍	新闻	科学文本	代码数据	其他数据
1174B	90B	134B	48B	100B	134B